

CHORUS: Complementary Experts for High-Coverage Testbench Stimulus Generation

Hejia Zhang¹, Sheng Lu², Zhongming Yu¹, Chia-Tung Ho³, Brucek Khailany³, Jishen Zhao¹

¹UC San Diego ²Georgia Institute of Technology ³NVIDIA

hez024@ucsd.edu, slu375@gatech.edu, zhy025@ucsd.edu, chiatungh@nvidia.com, bkhailany@nvidia.com, jzhao@ucsd.edu

Abstract

Large language models (LLMs) have advanced code generation, where executable feedback provides a more reliable learning signal than textual imitation alone. Hardware verification is an important application of code generation and accounts for a substantial fraction of modern chip design effort, with high-coverage testbench stimulus generation as a key task. We present CHORUS, a post-training framework that pushes performance beyond what a conventional supervised fine-tuning (SFT)-to-reinforcement learning (RL) pipeline achieves. CHORUS builds on two observations. First, staged SFT produces behaviorally diverse checkpoints, and dense-reward RL turns them into strong experts with comparable aggregate performance but distinct task-level strengths. Second, these complementary strengths can be exploited through either training-free model merging or further post-training to outperform the best individual expert. By consolidating the resulting specialists into a single 4B model, CHORUS achieves **88.0% Pass@1** on CVDP-ECov, outperforming DeepSeek-R1 (671B) by **13.5 percentage points**.

1 Introduction

Large language models (LLMs) have substantially advanced code generation, from producing short programs to solving complex tasks through compilation, execution, and iterative feedback. A central lesson from this progress is that executable feedback can provide a more reliable learning signal than textual imitation alone: generated programs can be run, evaluated, and improved according to their actual behavior. However, many specialized engineering tasks remain difficult even for frontier models, and simply increasing model size does not necessarily close the gap. This motivates better post-training methods that can extract more capability from compact, domain-specialized models.

Hardware verification is one such important coding application. Before a chip is manufactured, engineers must verify that its design behaves correctly across a wide range of operating conditions. A major part of this process is writing *testbenches*: executable programs that generate input stimuli, drive the design under verification, and measure which behaviors have been exercised. We focus specifically on *high-coverage testbench stimulus generation*, where the goal is to generate stimuli that maximize coverage when executed in a hardware simulator. This task differs from RTL design generation, assertion generation, or bug-specific checker synthesis:

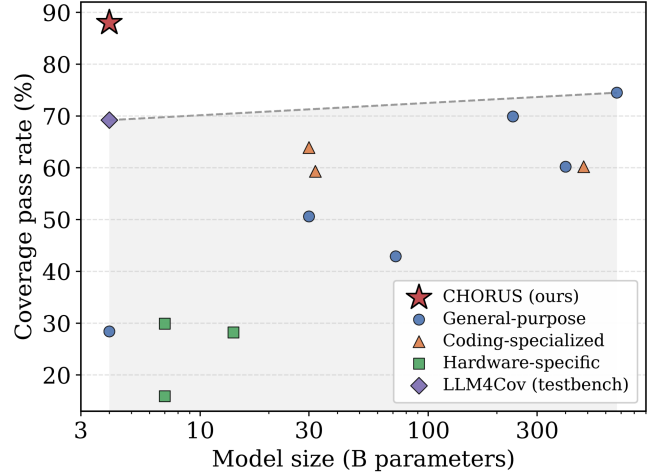


Figure 1: **Scale alone does not solve testbench generation.** Coverage pass rate versus model size on CVDP-ECov, with model size shown on a logarithmic scale. General-purpose, coding-specialized, and hardware-specific models follow only a weak size trend; even a 671B-parameter frontier model remains well below the best achievable performance. CHORUS (red star), a single 4B model, rises substantially above this frontier through targeted post-training.

the generated artifact is the stimulus program used to exercise an existing hardware design. Its quality is determined by a measurable, relatively dense, but non-differentiable execution signal – the coverage achieved after simulation.

As Figure 1 shows, scale alone is insufficient for this task. LLM4Cov (Zhang et al. 2026) improves a compact model through staged supervised fine-tuning (SFT), and a natural next step is to apply execution-guided reinforcement learning (RL) to its strongest final checkpoint. However, this conventional single-model pipeline eventually saturates. Rather than continuing to optimize one model, we ask whether the intermediate staged checkpoints can be transformed into multiple experts whose complementary strengths provide additional headroom beyond any individual model. Realizing this potential requires overcoming two challenges. First, **producing complementary experts**: the candidate models must be transformed into models that are not only individually

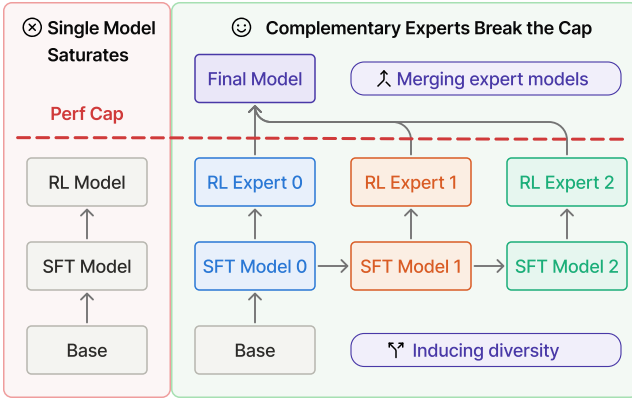


Figure 2: **Overview of CHORUS.** The conventional pipeline selects one SFT model, applies RL, and eventually saturates below a performance cap (left). CHORUS instead treats staged SFT as a source of complementary experts: it applies identical execution-guided RL to each staged-SFT checkpoint, then consolidates the resulting experts – through training-free merging or adaptive multi-teacher distillation – into a single model that surpasses the cap (right).

strong, but also retain distinct task-level capabilities despite being optimized for the same task. Second, **consolidating their complementary strengths**: these capabilities must be integrated into a single deployable model that outperforms every individual expert, without averaging away useful specialization or transferring inferior behavior.

We present CHORUS, a post-training framework that addresses both challenges, as illustrated in Figure 2. To **produce complementary experts**, CHORUS retains the intermediate checkpoints generated by staged SFT and applies the same execution-guided RL procedure to each one. Although these checkpoints begin with substantially different performance, RL turns them into experts with comparable aggregate accuracy. Crucially, the resulting models are not interchangeable: they continue to succeed on different subsets of tasks, and on the designs where they disagree the coverage gap between them is large. Staged SFT therefore provides more than a path toward one final checkpoint; together with dense execution-guided RL, it yields experts with complementary task-level strengths. To **consolidate complementary strengths**, we investigate two ways to convert this complementarity into a stronger single model. Training-free weight merging provides an immediate improvement over the best individual expert. We further introduce adaptive multi-teacher on-policy distillation, which selects task-specific teachers according to execution reward and skips tasks where no expert provides a superior solution. Both approaches surpass the single-expert saturation point. The final CHORUS model has only 4B parameters yet achieves 88.0% Pass@1 on CVDP-ECov, outperforming DeepSeek-R1 (671B) by 13.5 percentage points.

2 Background and Related Work

2.1 Post-Train for Code Generation

Execution-guided RL for code. RL with execution-based, verifiable rewards has become standard for code generation and reasoning, including GRPO and DeepSeekMath (Shao et al. 2024), DeepSeek-R1 (Guo et al. 2025), and DAPO (Yu et al. 2025), alongside code-specific methods such as CodeRL (Le et al. 2022) and RLEF (Gehring et al. 2025). Our RL stage follows the DAPO recipe used by CodeV-R1 (Zhu et al. 2025). Our contribution is not the RL recipe itself, but the observation that, in this setting, the SFT stage barely changes the eventual RL performance while leaving behind complementary experts.

Model merging. Averaging independently fine-tuned weights can improve accuracy without additional training (Wortsman et al. 2022). This idea has been extended through task arithmetic (Ilharco et al. 2023), Fisher-weighted merging (Matena and Raffel 2022), and interference-aware methods such as TIES (Yadav et al. 2023), DARE (Yu et al. 2024), and DELLA (Deep, Bhardwaj, and Poria 2024); see the survey of Yang et al. 2025. In the RL setting, weight-averaged policies and reward models, including Rewarded Soups, WARM, and WARP (Rame et al. 2023; Ramé et al. 2024; Ramé et al. 2024), as well as self-improvement followed by merging (Yuan et al. 2025), show that diversity across independent runs can be recovered through weight averaging. We use merging as an indicator of exploitable diversity. Crucially, prior work generally merges models that are *known a priori* to be heterogeneous, whereas we identify staged SFT as the *source* of heterogeneity that survives independent RL.

Distillation from multiple experts. On-policy distillation (Agarwal et al. 2024; Gu et al. 2024) trains a student on its own rollouts to reduce the train–inference mismatch of teacher-forced knowledge distillation (Hinton, Vinyals, and Dean 2015; Rusu et al. 2016). Multi-teacher knowledge distillation adaptively weights or selects teachers for each instance (Liu, Zhang, and Wang 2020; Yuan et al. 2021), while model-fusion methods combine heterogeneous LLMs (Wan et al. 2024; Jiang, Ren, and Lin 2023). Our adaptive OPD differs in both its teacher-selection signal and its gating behavior. It routes each task to the teacher that most outperforms the current student according to execution reward, and skips tasks where no teacher is superior. The student is therefore updated only where a better target demonstrably exists.

2.2 Background: Testbench Coverage

Given a hardware design under verification – the design sources together with a verification environment, including the module interface, reference or expected behavior, and compilation and simulation harness – the model must produce a *testbench* that generates input stimuli, drives the design, and records coverage over its signals and branches. A candidate testbench y for design x is compiled and run through an industrial simulator, which returns an execution status (compiles, simulates, or fails), a coverage fraction $c(x, y) \in [0, 1]$, and a log. This feedback is the only reliable

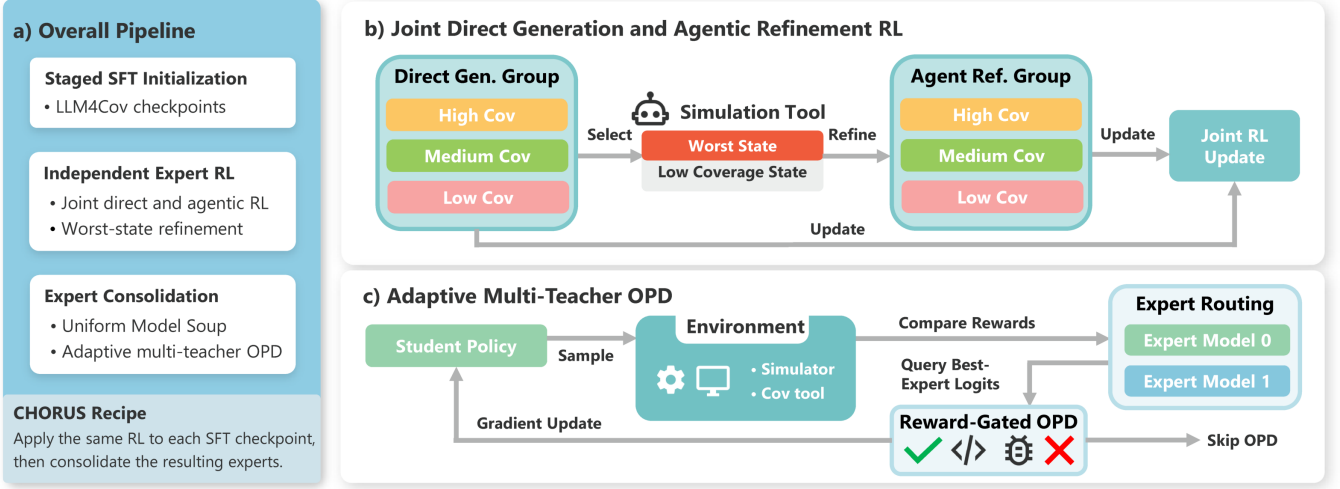


Figure 3: **The CHORUS pipeline.** (a) The three stages: staged-SFT initialization from LLM4Cov checkpoints, independent RL per checkpoint, and consolidation of the resulting experts by uniform Model Soup or adaptive multi-teacher OPD. (b) Execution-guided RL (DAPO) trains direct generation and agentic refinement jointly: the worst-coverage state in a sampled direct-generation group is selected, refined into an agentic-refinement group, and both groups feed one joint update. (c) Adaptive multi-teacher OPD compares execution rewards to route each task to its best expert and queries that expert’s logits; distillation is reward-gated, so a task with no better teacher is skipped rather than trained on.

measure of quality. Because it is *non-differentiable*, it rules out direct gradient supervision and motivates learning from the scalar execution outcome. Following LLM4Cov (Zhang et al. 2026), we work in an agentic setting. The model may either generate a testbench in a single pass (*direct generation*) or iterate by appending simulator feedback to its context and emitting a revised testbench for a bounded number of rounds (*agentic refinement*). Agentic refinement allows the model to react to execution signals that it could not anticipate from the design sources alone.

2.3 LLMs for hardware design and verification.

Most work on LLMs for hardware targets *design*, particularly the generation of RTL from natural-language specifications. Representative benchmarks include VerilogEval (Liu et al. 2023) and RTLLM (Lu et al. 2024); specialized models include RTLCoder (Liu et al. 2024), CodeV-R1 (Zhu et al. 2025), VeriCoder (Wei et al. 2025), and VeriReason (Wang et al. 2025b); and agentic systems include (Ho, Ren, and Khailany 2025; Zhao et al. 2025). A complementary line of work improves the *generalization* of hardware-code fine-tuning, for example through information-bottleneck regularization that limits memorization (Wang et al. 2025a).

In contrast, *verification* – including the generation of testbenches and stimuli – remains comparatively underexplored. Recent work includes AutoBench and CorrectBench (Qiu et al. 2024, 2025), and a concurrent line of work which reaches high pass rates by scaling *test-time* agentic search over a fixed, closed model (Yu et al. 2026). Those work and ours address complementary questions. They study how far inference-time scaffolding can push a frozen model, whereas we study how to *post-train* hardware-specific policies, including how the SFT curriculum shapes their RL outcomes

and preserves useful diversity. We conduct controlled, repeated post-training experiments on open 4B models, while the resulting insights concern the broader training recipe and may also inform post-training at larger scales.

The strongest prior system for our task and scenario is LLM4Cov (Zhang et al. 2026), which is our point of departure. LLM4Cov bootstraps its policy through a three-stage SFT curriculum: a warm-up stage that imitates full-teacher agentic traces, followed by two stages that synthesize traces under progressively more autonomous model configurations using worst-state-prioritized sampling. Under agentic evaluation, the resulting checkpoints establish state-of-the-art Pass@1 performance on CVD-ECov, its coverage-stimulus benchmark adapted from the CVD suite (Pinckney et al. 2025). We take these checkpoints, π_0, π_1, π_2 , as fixed starting points, leave the SFT procedure unchanged, and study what each contributes once execution-guided RL is applied.

3 Method

Figure 3 gives an overview of CHORUS. The pipeline takes the staged-SFT checkpoints of Zhang et al. (2026) as initializations and turns them into a single stronger model in three steps, corresponding to the three panels of the figure. First, we apply one identical execution-guided RL recipe to each checkpoint independently, training direct generation and agentic refinement under a single objective so that one policy serves both inference modes (Section 3.1, panel b). Running this recipe from the different SFT stages yields several RL experts that are comparable in aggregate accuracy but differ in *which* designs they solve (Section 3.2). The remaining question is how to collect those complementary strengths into one deployable model, and we pursue two answers. The training-

free route averages the experts’ weights (Section 3.3), which requires no additional compute but commits to one fixed combination for every task. The post-training route, adaptive multi-teacher on-policy distillation, instead keeps training a single student and lets the choice of teacher vary per task: it compares execution rewards to route each design to whichever expert is actually best on it, distills only when that expert beats the student, and otherwise skips the task (Section 3.4, panel c). Sections 3.1–3.4 describe each step in turn.

3.1 Joint Direct Generation and Agentic Refinement

We optimize a single policy π_θ to be good at *both* the direct and agentic-refinement modes of Section 2.2. Each training step mixes direct-generation and agentic-refinement rollout groups, jointly training the policy to generate strong testbenches from scratch and repair weak ones using feedback. For refinement we follow the worst-state–prioritized strategy of LLM4Cov: within a group we locate the *least*-covering candidate and continue refinement from that state, focusing optimization on the point in the trajectory the model handles worst rather than polishing already-successful rollouts.

Reward. The scalar reward is built directly from simulator feedback. For design x and candidate testbench y with coverage fraction $c(x, y)$,

$$R(x, y) = \begin{cases} 1 + c(x, y), & \text{if } y \text{ runs and yields coverage,} \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

so a perfectly covering testbench scores 2, any executing candidate scores at least 1 plus its coverage, and any non-compiling or non-simulating candidate scores 0 – cleanly separating “runs and covers” from “fails.”

Policy optimization. We optimize with DAPO (Yu et al. 2025), following the Verilog-generation recipe of CodeV-R1 (Zhu et al. 2025). Let a group of G trajectories $\{y_i\}$ be sampled for design x , with rewards $R(x, y_i)$ from Eq. (1) and group-relative advantages $\hat{A}_i = (R(x, y_i) - \text{mean}_j R(x, y_j)) / \text{std}_j R(x, y_j)$. Writing the per-token importance ratio $r_{i,t}(\theta) = \pi_\theta(y_{i,t} \mid x, y_{i,<t}) / \pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$, the objective is the token-level clipped surrogate

$$\mathcal{L}_{\text{DAPO}}(\theta) = -\mathbb{E} \left[\frac{1}{\sum_i |y_i|} \sum_{i,t} \min \left(r_{i,t} \hat{A}_i, \text{clip}(r_{i,t}, 1 - \varepsilon_{\text{lo}}, 1 + \varepsilon_{\text{hi}}) \hat{A}_i \right) \right]. \quad (2)$$

with asymmetric “Clip-Higher” bounds $\varepsilon_{\text{lo}} < \varepsilon_{\text{hi}}$ to encourage exploration, *no* KL penalty to the initialization, and no dynamic sampling. We keep the recipe deliberately standard; our contributions lie in what we do *around* it.

3.2 SFT-Initialized RL Experts

Applying the procedure of Section 3.1 independently to each staged-SFT checkpoint yields three *RL experts* $\pi_0^{\text{RL}}, \pi_1^{\text{RL}}, \pi_2^{\text{RL}}$, initialized from π_0, π_1, π_2 . The three runs share identical RL data, objective, and training budget; the

only difference is the SFT initialization. This isolates the effect of the SFT stage on the RL outcome and, as Section 5 shows, exposes the diversity our method exploits.

3.3 Exploiting Diversity through Model Merging

As a training-free way to combine the experts we consider weight-space merging. The simplest, *Model Soup* (Wortsman et al. 2022), averages the three experts’ parameters. We also evaluate interference-aware merges – TIES (Yadav et al. 2023) with DARE sparsification (Yu et al. 2024), and DELLA (Deep, Bhardwaj, and Poria 2024) – which sparsify and sign-align task vectors before combining. Merging costs no additional training and, because the experts began from a shared SFT lineage, their parameters remain mergeable; it serves as both a strong baseline and a first probe of how much of the experts’ complementary skill lives in a linearly combinable subspace.

3.4 Adaptive Multi-Teacher OPD

Merging combines the experts once and for all in weight space. Our main method instead combines them *adaptively, per task*: it keeps training a student policy π_θ (initialized from one RL expert), but on each task it may learn from whichever expert is actually better *on that task*, and otherwise leaves that task out of the update. The experts $\{\pi_t\}$ act as a pool of teachers.

Reward-gated teacher routing. For a design x , each teacher and the student produce rollouts scored by the simulator reward (Eq. (1)). Let $t^* = \arg \max_t R(\pi_t, x)$ be the best-performing teacher on x . We distill from t^* only when it genuinely beats the current student; otherwise the task contributes no gradient:

$$\mathcal{L}(x) = \begin{cases} \mathcal{L}_{\text{OPD}}(x; \pi_{t^*}), & R(\pi_{t^*}, x) > R(\pi_\theta, x), \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Two properties matter. First, routing is *dynamic and online*: the teacher is selected separately for each design according to its current rollout reward, rather than assigned in advance based on a fixed task type. The student therefore learns from whichever expert is strongest on each instance, allowing it to inherit complementary strengths that need not follow a predefined task partition. Second, the reward gate prevents the student from being dragged toward a teacher that is worse than itself on a task; when no teacher beats the student, the task is simply *skipped* rather than trained on, since the student is already at least as good there. Within a training batch, only the gated distillation examples contribute to the update.

Distillation loss. For the distillation term we use a variance-reduced on-policy distillation objective (v-OPD) (Oh et al. 2026): the student is updated on its *own* rollouts toward the teacher via a sampled-token reverse-KL signal, with a detached control variate computed over the student’s top- K vocabulary support to reduce gradient variance. Distilling on execution-grounded rollouts rather than teacher-forced imitation keeps the student stable while it absorbs the teachers; we adopt the objective as a tool and give its exact form in Appendix C.

Type	Model	CVDV-ECov				AutoEval-ECov			
		Agentic		Direct Infer		Agentic		Direct Infer	
		Pass@1	Cov@1	Pass@1	Cov@1	Pass@1	Cov@1	Pass@1	Cov@1
General-Purpose	DeepSeek-R1 (671B)	74.5%	88.5%	32.5%	51.7%	92.4%	98.7%	69.4%	87.9%
	Llama-4-Maverick (400B)	60.2%	81.7%	23.1%	47.3%	32.9%	48.7%	21.4%	41.9%
	Qwen3-235B-A22B	69.9%	83.2%	29.9%	47.0%	84.0%	92.6%	70.6%	83.4%
	Qwen2.5-72B	42.9%	64.1%	16.4%	32.2%	63.8%	87.6%	20.9%	73.3%
	Qwen3-30B-A3B	50.6%	68.0%	30.1%	52.2%	81.9%	91.6%	72.3%	86.1%
Coding-Specialized	Qwen2.5-Coder-32B	59.3%	79.6%	23.4%	52.1%	74.5%	88.0%	52.1%	80.1%
	Qwen3-Coder-30B-A3B	63.9%	79.9%	28.4%	48.6%	83.8%	94.0%	75.3%	87.1%
	Qwen2.5-Coder-7B	20.5%	34.4%	11.3%	26.6%	59.9%	77.5%	20.9%	52.2%
Hardware / Verification	VeriCoder-Qwen2.5-14B	28.2%	47.9%	17.1%	37.3%	54.0%	69.2%	28.3%	56.2%
	CodeV-R1-RL-Qwen-7B	29.9%	55.5%	14.5%	32.5%	68.5%	85.6%	42.6%	67.9%
	VeriReason-Qwen2.5-7B	15.9%	26.4%	8.9%	16.2%	41.5%	55.0%	13.7%	27.9%
	CorrectBench (235B)	34.9%	60.5%	–	–	55.8%	84.9%	–	–
	LLM4Cov-Qwen3-4B	69.2%	90.4%	50.6%	76.4%	85.0%	96.3%	79.6%	92.6%
CHORUS (ours, 4B)	RL on Best SFT	85.3%	95.5%	74.9%	87.4%	88.5%	97.0%	86.9%	95.6%
	Merge (training-free)	86.7%	94.9%	79.5%	89.4%	88.7%	97.2%	87.2%	95.5%
	Merge (post-training)	88.0%	96.0%	<u>78.8%</u>	<u>88.7%</u>	<u>89.0%</u>	97.0%	85.1%	94.5%

Table 1: Main results following the Pass Rate / Avg. Coverage protocol of LLM4Cov, reported as Pass@1 / Cov@1. **Bold** marks the best and underline the second-best entry in each column. CorrectBench is a multi-agent system with no direct-inference result, and its size refers to its backbone we used in evaluation; LLM4Cov-Qwen3-4B is its strongest staged-SFT checkpoint (Stage-2). “RL on Best SFT” applies our RL stage to the strongest staged-SFT checkpoint; the two merge rows consolidate the three RL experts, training-free (Model Soup) or through adaptive multi-teacher OPD.

4 Experimental Setup

Benchmarks and metrics. We evaluate on the two benchmarks of Zhang et al. (2026). Our primary benchmark is **CVDV-ECov**, 83 hardware repositories adapted from the CVDV suite (Pinckney et al. 2025), where the coverage threshold for a task is set by human experts. We additionally report **AutoEval-ECov**, 156 tasks derived from VerilogEval (Liu et al. 2023) following the CorrectBench methodology (Qiu et al. 2025), whose threshold is the stricter requirement of 100% coverage. We report Pass@1 and Pass@5 – the fraction of tasks whose coverage exceeds the threshold, using a single sample or the best of 5 – and Coverage@1/5, the mean coverage under the same sampling, scoring a failed simulation as 0% coverage. Both benchmarks are evaluated in the two modes of Section 2.2: *agentic* refinement and single-pass *direct inference*. Unless stated otherwise we evaluate under agentic refinement with $N=3$ rounds, $n=5$ samples per task, generation temperature 0.7, and top- p 0.8. We report all of these for completeness, but they are not of equal weight: our headline metric is Pass@1 on CVDV-ECov under agentic refinement, since Pass@1 is the deployment-relevant quantity – a testbench either reaches the coverage bar or it does not – and coverage, direct inference, and AutoEval-ECov are reported as supporting evidence.

Models and initialization. All policies are 4B-parameter models. The three RL experts are initialized from the stage-0/1/2 SFT checkpoints of Zhang et al. (2026) and trained with the identical RL configuration of Section 3.1.

Training. As in LLM4Cov, we use the hardware-repository dataset introduced by CodeV-R1 for post-training. We use DAPO with asymmetric clipping ($\varepsilon_{lo}=0.2$, $\varepsilon_{hi}=0.28$), no KL penalty, a constant learning rate of 1×10^{-6} , and group sampling with direct/refinement rollouts. We take 1000 RL steps as the reported operating point – an a-priori budget at which Pass@1 has saturated (Section 5.1) – rather than selecting a step by test-set score. Adaptive OPD continues from an RL expert for a further 100 steps, again using an a-priori operating point. Merges are computed post hoc from the three experts. Full hyperparameters and hardware details are provided in Appendix A and Appendix B.

5 Results and Analysis

Headline result. Table 1 places CHORUS against general-purpose, coding, and hardware-specific baselines under the agentic protocol of Zhang et al. (2026). On our primary metric, Pass@1 on CVDV-ECov under agentic refinement, our best single 4B model reaches 88.0%, outperforming the 671B DeepSeek-R1 by 13.5 points and the prior state of the art, LLM4Cov Stage-2, by a wider margin still. It leads every general-purpose, coding, and hardware/verification baseline we evaluate. The secondary axes of Table 1 agree: the ordering is unchanged under direct inference, which our RL stage trains jointly with refinement, and under the coverage metric. The one place a baseline leads is DeepSeek-R1 on AutoEval-ECov, whose 156 single-module designs are small enough for a 671B reasoning model with a long output budget to solve directly; that advantage does not carry to the

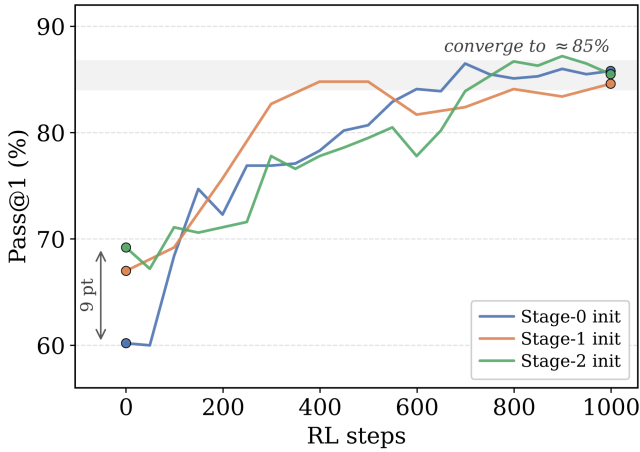


Figure 4: **Pass@1 versus RL steps for the three SFT initializations.** The 9-percentage-point spread at step 0 collapses under identical RL: all three converge to $\approx 85\%$. The weakest start (Stage-0) is not the weakest endpoint.

larger CVDP-ECov designs. The rest of this section explains *how* these performance gains are achieved and *why* the combination step is essential to unlocking them.

5.1 Does Staged SFT Improve the RL Optimum?

We first examine Table 1 stage by stage. Before RL the three SFT checkpoints span a 9-point Pass@1 range, exactly the gradient the staged curriculum is designed to produce. After identical RL, that gradient is gone: the three experts land within roughly a point of one another, and the stage-0 expert, the weakest initialization, is no longer the weakest endpoint. Figure 4 shows the full trajectories: the curves start far apart and interleave into a common $\approx 85\%$ band well before the 1000-step operating point, with coverage saturating even earlier. The practical reading is blunt – *the later, more elaborate SFT stages buy almost nothing once execution-guided RL is applied*. This is the observation that makes the rest of the paper interesting: if the stages are redundant for the RL optimum, why keep them at all?

5.2 Are the Converged Experts Complementary?

Equal aggregate accuracy need not mean equal behavior. We find the three experts are in fact strongly complementary.

Aggregate evidence. Per Figure 5, although the experts score within about a point of one another individually, they succeed on *different* designs. Taking the oracle union, counting a design as solved if *any* expert solves it, reaches 90.8% Pass@1, roughly 5 points above the best single expert. This ≈ 5 -point gap between any single expert and their union is the *headroom* that the rest of the paper tries to recover: it is achievable in principle, because the experts already collectively solve those designs.

Per-design evidence. The aggregate gap summarizes the disagreement; the per-design view shows its shape. Most designs are solved (or missed) by all three experts, but a meaningful set is split, and where the experts disagree most the

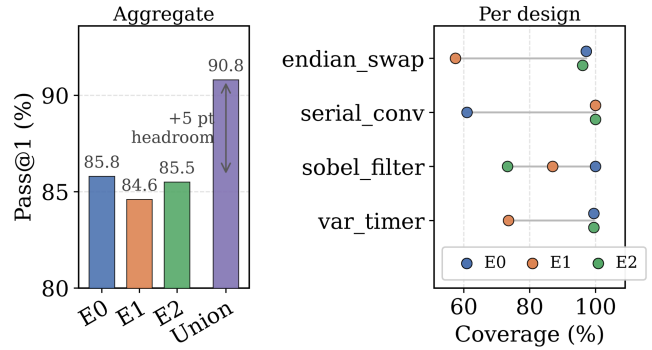


Figure 5: **The converged experts are complementary.** Left: each expert scores similar Pass@1 alone, but their oracle union leads with ≈ 5 points of headroom. Right: per-design coverage on the four CVDP-ECov designs where the experts disagree most. No expert dominates.

Method	Pass@1	Pass@5
<i>Reference</i>		
Best individual expert	85.8%	90.4%
<i>Training-free merges</i>		
Model Soup	86.7%	91.6%
Best DARE-TIES	86.0%	92.8%
Best DELLA	85.8%	90.4%
<i>Upper bound (not a trained model)</i>		
Oracle union of experts	90.8%	92.8%

Table 2: Training-free merging on CVDP-ECov. Simple averaging gives a real gain over the best single expert; interference-aware merges are not reliably better. The oracle union is a *theoretical* upper bound – a design counts as solved if *any* expert solves it – and shows substantial diversity that static merging leaves unrecovered.

coverage spread reaches tens of points on the same design. Crucially the leader rotates: an expert that nearly saturates one design can fall to little more than half coverage on the next, where a sibling saturates instead. That rotation makes the headroom exploitable: if one expert dominated everywhere the union would collapse onto that expert, leaving nothing to combine, whereas a method that picks the right expert per design has something real to recover.

5.3 Can Training-Free Merging Exploit the Diversity?

Given complementary experts, the first question is whether a training-free weight merge can turn that complementarity into accuracy. Table 2 shows it partly can. A uniform Model Soup of the three experts lands about a point above the best individual expert and clearly above the average expert. As such, some of the complementary skill does live in a linearly combinable subspace. But the more elaborate interference-aware merges are not reliably better: DARE-TIES improves Pass@5 but not Pass@1, and both TIES and DELLA are

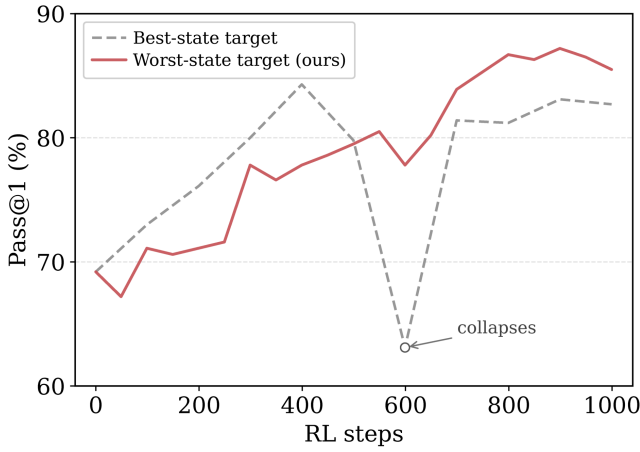


Figure 6: **Worst-state refinement targets are more stable than best-state ones.** Identical RL from the Stage-2 checkpoint on CVDP-ECov, varying only which state in a sampled group is refined. Best-state selection rises faster early but destabilizes mid-run and ends lower; worst-state selection (ours) climbs steadily and finishes higher.

Method	Ungated tasks	Pass@1
RL on Best SFT (start)	–	85.3%
Continued pure RL	–	85.1%
Best Training-free	–	86.7%
Always distill (best teacher)	distill	86.3%
Reward-gated + RL fallback	train (RL)	87.5%
Adaptive OPD (ours)	skip	88.0%

Table 3: Adaptive-OPD ablations at same 100-step operating point, continuing from the RL expert on the best SFT stage. The middle column states what each variant does with a task whose best teacher does not beat the student.

sensitive to which expert is used as the base (full variants in Appendix D). Such observation suggests exploring methods that can adapt to *which* expert is right for *which* task.

5.4 Can Adaptive OPD Exploit the Diversity?

Our adaptive multi-teacher OPD (Section 3.4) continues training a student, i.e., the RL expert on the best SFT stage, while routing each design to its best teacher under the reward gate. Within its 100-step operating window it reaches 88.0% Pass@1, roughly three points above the starting expert and above the best static merge, recovering over half of the oracle-union headroom (Table 3). The comparison that isolates the mechanism is *continued pure RL* from the same checkpoint: with the teachers removed, further RL does not improve the student and, if anything, drifts slightly down. The improvement therefore comes from the teachers’ complementary knowledge, not from simply training longer.

5.5 Other Ablations

Adaptive OPD differs from “just distill from a few checkpoints” in two design choices, and Table 3 is built to isolate them. **(1) The reward gate.** *Always distilling* from the current best teacher (even on tasks where that teacher is no better than the student) drags the student toward mediocre targets, and recovers only about a third of what the full method obtains from the same teachers. **(2) Skip vs. fall back.** On tasks with *no* superior teacher, one could still train, e.g., fall back to the ordinary DAPO objective, but given our student has already converged on RL learning, further DAPO on itself may distract it from teacher supervision. The outer anchors bound the overall effect: with the teachers removed entirely, *continued pure RL* does not improve the student at the same budget, whereas the full method reaches 88.0%. That gap measures what reward-gated, skip-when-unbeaten distillation buys. Both design choices are therefore load-bearing. In that order, the gate accounts for most of the gain, the skip rule for the remainder.

Which state to refine. A separate ablation supports the RL stage’s refinement design (Figure 6). Selecting the *worst*-coverage state in a sampled group as the refinement target is not just better at the operating point than selecting the best. It is far more stable. Best-state selection improves faster over the first few hundred steps, which is unsurprising: refining an already-good testbench is an easier problem, and the reward signal is cleaner. But it then collapses by over twenty points mid-run before partially recovering, and never regains its own earlier peak. We read this as a coverage-distribution effect: refining the best state concentrates training on states the policy has already mastered, so the gradient carries little new information and the policy is free to drift, whereas the worst state is where coverage is actually missing and therefore where the execution signal is most informative. Targeting the hardest state in each group makes each update earn its keep, turning a volatile run into a monotone one.

6 Limitations and Conclusion

Limitations. Our study is confined to hardware testbench generation; while we use two benchmarks, they are one application family, and whether staged SFT induces the same durable diversity in unrelated RL domains is an open question we do not settle here. Additionally, the diversity we exploit originates in a specific SFT curriculum (Zhang et al. 2026); other curricula may induce more or less of it.

Conclusion. We introduced CHORUS, a post-training framework that turns related SFT checkpoints into complementary RL experts and consolidates their strengths into one model. Although the experts converge to similar overall performance, they retain distinct task-level capabilities. Training-free merging captures part of this complementarity, while adaptive multi-teacher OPD improves further by routing each task to its strongest expert and skipping updates when no teacher is better. The resulting 4B model reaches 88.0% Pass@1 on CVDP-ECov, showing that consolidating complementary experts can push performance beyond single-model RL saturation.

References

- Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos Garea, S.; Geist, M.; and Bachem, O. 2024. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; and Sun, Y., eds., *International Conference on Learning Representations*, volume 2024, 21246–21263.
- Deep, P. T.; Bhardwaj, R.; and Poria, S. 2024. DELLA-Merging: Reducing Interference in Model Merging through Magnitude-Based Sampling. arXiv:2406.11617.
- Gehring, J.; Zheng, K.; Copet, J.; Mella, V.; Cohen, T.; and Synnaeve, G. 2025. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 19034–19055. PMLR.
- Gu, Y.; Dong, L.; Wei, F.; and Huang, M. 2024. MiniLLM: Knowledge Distillation of Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081): 633–638.
- Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. In *NIPS Deep Learning and Representation Learning Workshop*.
- Ho, C.-T.; Ren, H.; and Khailany, B. 2025. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 300–307.
- Ilharco, G.; Ribeiro, M. T.; Wortsman, M.; Schmidt, L.; Hajishirzi, H.; and Farhadi, A. 2023. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*.
- Jiang, D.; Ren, X.; and Lin, B. Y. 2023. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In Rogers, A.; Boyd-Graber, J.; and Okazaki, N., eds., *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 14165–14178. Toronto, Canada: Association for Computational Linguistics.
- Le, H.; Wang, Y.; Gotmare, A. D.; Savarese, S.; and Hoi, S. C. H. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 21314–21328. Curran Associates, Inc.
- Liu, M.; Pinckney, N.; Khailany, B.; and Ren, H. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 1–8.
- Liu, S.; Fang, W.; Lu, Y.; Zhang, Q.; Zhang, H.; and Xie, Z. 2024. RTLcoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution. In *2024 IEEE LLM Aided Design Workshop (LAD)*, 1–5.
- Liu, Y.; Zhang, W.; and Wang, J. 2020. Adaptive multi-teacher multi-level knowledge distillation. *Neurocomputing*, 415: 106–113.
- Lu, Y.; Liu, S.; Zhang, Q.; and Xie, Z. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 722–727.
- Matena, M.; and Raffel, C. 2022. Merging models with fisher-weighted averaging. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22. Red Hook, NY, USA: Curran Associates Inc. ISBN 9781713871088.
- Oh, M.; Song, S.; Choi, G.; Choi, Y.; and Jo, Y. 2026. KL for a KL: On-Policy Distillation with Control Variate Baseline. arXiv:2605.07865.
- Pinckney, N.; Deng, C.; Ho, C.-T.; Tsai, Y.-D.; Liu, M.; Zhou, W.; Khailany, B.; and Ren, H. 2025. Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification. arXiv:2506.14074.
- Qiu, R.; Zhang, G. L.; Drechsler, R.; Schlichtmann, U.; and Li, B. 2024. AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD, MLCAD ’24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400706998.
- Qiu, R.; Zhang, G. L.; Drechsler, R.; Schlichtmann, U.; and Li, B. 2025. CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design. In *2025 Design, Automation & Test in Europe Conference (DATE)*, 1–7.
- Rame, A.; Couairon, G.; Dancette, C.; Gaya, J.-B.; Shukor, M.; Soulier, L.; and Cord, M. 2023. Rewarded soups: towards Pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 71095–71134. Curran Associates, Inc.
- Ramé, A.; Vieillard, N.; Hussenot, L.; Dadashi, R.; Cideron, G.; Bachem, O.; and Ferret, J. 2024. WARM: on the benefits of weight averaged reward models. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org.
- Ramé, A.; Ferret, J.; Vieillard, N.; Dadashi, R.; Hussenot, L.; Cedoz, P.-L.; Sessa, P. G.; Girgin, S.; Douillard, A.; and Bachem, O. 2024. WARP: On the Benefits of Weight Averaged Rewarded Policies. arXiv:2406.16768.
- Rusu, A. A.; Colmenarejo, S. G.; Gülçehre, Ç.; Desjardins, G.; Kirkpatrick, J.; Pascanu, R.; Mnih, V.; Kavukcuoglu, K.;

- and Hadsell, R. 2016. Policy Distillation. In Bengio, Y.; and LeCun, Y., eds., *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Wan, F.; Huang, X.; Cai, D.; Quan, X.; Bi, W.; and Shi, S. 2024. Knowledge Fusion of Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- Wang, C.; Chen, X.; Liu, S.; and Ding, K. 2025a. Breaking Memorization Barriers in LLM Code Fine-Tuning via Information Bottleneck for Improved Generalization. arXiv:2510.16022.
- Wang, Y.; Sun, G.; Ye, W.; Qu, G.; and Li, A. 2025b. VeriReason: Reinforcement Learning with Testbench Feedback for Reasoning-Enhanced Verilog Generation. arXiv:2505.11849.
- Wei, A.; Tan, H.; Suresh, T.; Mendoza, D.; Teixeira, T. S. F. X.; Wang, K.; Trippel, C.; and Aiken, A. 2025. VeriCoder: Enhancing LLM-Based RTL Code Generation through Functional Correctness Validation. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.
- Wortsman, M.; Ilharco, G.; Gadre, S. Y.; Roelofs, R.; Gontijo-Lopes, R.; Morcos, A. S.; Namkoong, H.; Farhadi, A.; Carmon, Y.; Kornblith, S.; and Schmidt, L. 2022. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In Chaudhuri, K.; Jegelka, S.; Song, L.; Szepesvari, C.; Niu, G.; and Sabato, S., eds., *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, 23965–23998. PMLR.
- Yadav, P.; Tam, D.; Choshen, L.; Raffel, C.; and Bansal, M. 2023. TIES-Merging: Resolving Interference When Merging Models. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 7093–7115. Curran Associates, Inc.
- Yang, E.; Shen, L.; Guo, G.; Wang, X.; Cao, X.; Zhang, J.; and Tao, D. 2025. Model Merging in LLMs, MLLMs, and Beyond: Methods, Theories, Applications and Opportunities. arXiv:2408.07666.
- Yu, C.; Deng, C.; Pinckney, N.; and Khailany, B. 2026. Agentic Hardware Design as Repository-Level Code Evolution. arXiv:2606.28279.
- Yu, L.; Yu, B.; Yu, H.; Huang, F.; and Li, Y. 2024. Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch. In Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; and Berkenkamp, F., eds., *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 57755–57775. PMLR.
- Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, J.; Liu, L.; Liu, X.; Lin, H.; Lin, Z.; Ma, B.; Sheng, G.; Tong, Y.; Zhang, C.; Zhang, M.; Zhang, R.; Zhang, W.; Zhu, H.; Zhu, J.; Chen, J.; Chen, J.; Wang, C.; Yu, H.; Song, Y.; Wei, X.; Zhou, H.; Liu, J.; Ma, W.-Y.; Zhang, Y.-Q.; Yan, L.; Wu, Y.; and Wang, M. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In Belgrave, D.; Zhang, C.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi, M.; and Chen, N., eds., *Advances in Neural Information Processing Systems*, volume 38, 113222–113244. Curran Associates, Inc.
- Yuan, F.; Shou, L.; Pei, J.; Lin, W.; Gong, M.; Fu, Y.; and Jiang, D. 2021. Reinforced multi-teacher selection for knowledge distillation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, 14284–14291.
- Yuan, X.; Zhang, C.; Liu, Z.; Shi, D.; Pan, L.; Vosoughi, S.; and Lee, W. 2025. Superficial Self-Improved Reasoners Benefit from Model Merging. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 5901–5921. Suzhou, China: Association for Computational Linguistics. ISBN 979-8-89176-332-6.
- Zhang, H.; Yu, Z.; Ho, C.-T.; Ren, H.; Khailany, B.; and Zhao, J. 2026. LLM4Cov: Execution-Aware Agentic Learning for High-Coverage Testbench Generation. ICML 2026, arXiv:2602.16953.
- Zhao, Y.; Zhang, H.; Huang, H.; Yu, Z.; and Zhao, J. 2025. MAGE: A Multi-Agent Engine for Automated RTL Code Generation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 1–7.
- Zhu, Y.; Huang, D.; Lyu, H.; Zhang, X.; Li, C.; Shi, W.; Wu, Y.; Mu, J.; Wang, J.; Zhao, Y.; Jin, P.; Cheng, S.; Liang, S.; Zhang, X.; Zhang, R.; Du, Z.; Guo, Q.; Hu, X.; and Chen, Y. 2025. QiMeng-CodeV-R1: Reasoning-Enhanced Verilog Generation. In Belgrave, D.; Zhang, C.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi, M.; and Chen, N., eds., *Advances in Neural Information Processing Systems*, volume 38, 154266–154300. Curran Associates, Inc.

A Dataset and Evaluation Settings

Post-training dataset. All RL and OPD runs use the 11,488-record `train` split of `CodeV-R1-11kRTL`, derived from `CodeV-R1` revision `ffc469807109`. We retain designs with more than 1,000 RTL tokens and exactly one candidate top module. This selection focuses post-training on complex RTL problems while avoiding ambiguous top-level designs. Each record contains `question`, `problem_id`, `ground_truth`, and `rl_response`.

Evaluation benchmarks. We follow the LLM4Cov protocol. CVDV-ECov contains 83 hardware repositories with per-repository human-expert coverage thresholds. AutoEval-ECov contains 156 VerilogEval-derived tasks and requires 100% coverage.

Metrics. Let M be the number of benchmark tasks and $N = 5$ the number of independently generated samples per task. For task i , sample j , achieved coverage $c_{ij} \in [0, 1]$, and task threshold τ_i , we compute

$$\text{Pass@1} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N \mathbf{1}[c_{ij} \geq \tau_i], \quad \text{Pass@5} = \frac{1}{M} \sum_{i=1}^M \max_{1 \leq j \leq N} \mathbf{1}[c_{ij} \geq \tau_i], \quad (4)$$

$$\text{Cov@1} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N c_{ij}, \quad \text{Cov@5} = \frac{1}{M} \sum_{i=1}^M \max_{1 \leq j \leq N} c_{ij}. \quad (5)$$

Invalid compilation, simulation, or coverage reports receive zero coverage. Thus, @1 averages the five samples, whereas @5 takes the best sample for each task. The headline metric is CVDV-ECov agentic Pass@1.

Evaluation settings. Agentic evaluation uses three interaction rounds, 5 samples per task, temperature 0.7, and top- p 0.8. Our Qwen3-4B models use a 16,384-token response cap. For API-served baselines such as DeepSeek-R1, we use the model-specific maximum response length exposed by the Google API.

EDA settings. All hardware simulations and coverage evaluations are performed using Cadence Xcelium and IMC toolchains on a Rocky Linux 8.9 environment. We use `xrun` (version 22.03-s001) as the SystemVerilog simulator for compilation and execution, and Cadence IMC (version 25.09-a001) for post-simulation coverage analysis.

B RL DAPO Training Detailed Settings and Results

B.1 Initializers and Hyperparameter

The released LLM4Cov Qwen3-4B Stage-0, Stage-1, and Stage-2 SFT checkpoints from the `hez2024` Hugging Face collection serve as fixed initializers. Each 1000-update RL run uses $2 \times$ NVIDIA H100 PCIe GPUs, takes about 50 wall-clock hours, and consumes about 100 GPU-hours.

Category	Value	Category	Value
Rollout seed	42	Dynamic sampling	disabled
Prompt batch size	2	Optimizer	Adam
Agentic rounds	2	Learning rate	10^{-6}
Samples per prompt	4	Learning-rate schedule	constant
Rollout batch size	4	Weight decay	0.0
Global batch size	16	Adam beta 1	0.9
Response cap	16,384 tokens	Adam beta 2	0.95
Packed-token cap	24,576 tokens per GPU	Tensor parallel size	2
Rollout temperature	1.0	Pipeline parallel size	1
Evaluation temperature	0.7	Context parallel size	1
Evaluation top- p	0.8	Sequence parallelism	enabled
Advantage estimator	GRPO	Dynamic batching	enabled
Loss granularity	per-token	Rollout engine	SGLang
DAPO lower clip	$\epsilon_{lo} = 0.2$	GPUs per rollout engine	2
DAPO upper clip	$\epsilon_{hi} = 0.28$	Static memory fraction	0.35
KL coefficient	0.0	Training EDA feedback	enabled
Entropy coefficient	0.0	Evaluation EDA feedback	enabled

Table 4: Final RL configuration. The same configuration is applied independently to all three SFT initializers.

B.2 Results

Table 5 reports all three RL trajectories at 100-update boundaries; step 0 is the SFT initializer.

Step	Stage-0				Stage-1				Stage-2			
	Pass@1	Pass@5	Cov@1	Cov@5	Pass@1	Pass@5	Cov@1	Cov@5	Pass@1	Pass@5	Cov@1	Cov@5
0	60.2	77.1	82.1	94.9	67.0	84.3	88.2	95.3	69.2	81.9	90.4	96.1
100	68.4	84.3	87.1	95.7	69.2	84.3	89.2	96.5	71.1	86.7	89.3	96.7
200	72.3	83.1	90.3	97.1	75.7	86.7	92.1	96.8	71.1	85.5	91.6	96.9
300	76.9	90.4	90.1	97.6	82.7	90.4	95.1	97.2	77.8	88.0	92.0	97.1
400	78.3	90.4	90.2	96.6	84.8	91.6	95.0	96.7	77.8	88.0	92.1	96.6
500	80.7	89.2	92.5	97.6	84.8	90.4	94.8	96.3	79.5	86.7	93.3	97.1
600	84.1	91.6	93.7	97.4	81.7	89.2	94.3	96.1	77.8	86.7	90.6	95.1
700	86.5	91.6	94.6	97.8	82.4	90.4	94.5	96.7	83.9	92.8	94.8	97.9
800	85.1	92.8	95.2	97.9	84.1	94.0	93.9	98.0	86.7	90.4	95.0	97.4
900	86.0	89.2	96.5	97.9	83.4	91.6	94.4	97.6	87.2	91.6	95.6	97.4
1000	85.8	89.2	95.8	97.5	84.6	90.4	95.8	97.6	85.3	89.2	95.5	97.1

Table 5: CVDP-ECov RL-DAPO trajectories for Stage-0, Stage-1, and Stage-2; all metrics are percentages.

B.3 Worst-State versus Best-State Refinement

We vary only the rollout used as the next-round agentic-refinement state in Stage-2 RL. The main configuration selects the lowest-coverage state, whereas the ablation selects the highest-coverage state; all other hyperparameters remain fixed. Best-state refinement improves faster initially, but drops sharply at step 600 and finishes below worst-state refinement in Pass@1.

Step	Worst-state (ours)				Best-state			
	Pass@1	Pass@5	Cov@1	Cov@5	Pass@1	Pass@5	Cov@1	Cov@5
0	69.2	81.9	90.4	96.1	69.2	81.9	90.4	96.1
100	71.1	86.7	89.3	96.7	73.0	84.3	91.3	97.4
200	71.1	85.5	91.6	96.9	76.1	90.4	92.5	95.9
300	77.8	88.0	92.0	97.1	80.0	91.6	92.7	97.0
400	77.8	88.0	92.1	96.6	84.3	91.6	94.5	97.1
500	79.5	86.7	93.3	97.1	79.8	89.2	93.2	97.4
600	77.8	86.7	90.6	95.1	63.1	85.5	69.6	93.8
700	83.9	92.8	94.8	97.9	81.4	90.4	93.0	96.4
800	86.7	90.4	95.0	97.4	81.2	91.6	92.2	97.4
900	87.2	91.6	95.6	97.4	83.1	91.6	94.4	97.4
1000	85.3	89.2	95.5	97.1	82.7	91.6	95.6	96.2

Table 6: CVDP-ECov refinement-target ablation for Stage-2 RL; all metrics are percentages. Figure 6 plots the Pass@1 trajectories; this table provides all four metrics at 100-update boundaries.

C Adaptive Multi-Teacher OPD Detailed Settings and Results

C.1 Routing and v-OPD Objective

The student is stage-2 RL at step 1000; the teachers are stage-0 and stage-1 RL at step 1000. The student generates 4 candidates per prompt round and each teacher generates 2. Let π_{t^*} denote the teacher selected by the routing rule. For a student-sampled token y_t with context c_t , define the detached per-token OPD reward

$$r_t = \log \pi_{t^*}(y_t | c_t) - \log \pi_\theta(y_t | c_t). \quad (6)$$

Let S_t be the $K = 16$ most likely tokens under the student and let $\bar{\pi}_\theta, \bar{\pi}_{t^*}$ be the student and routed-teacher distributions renormalized on S_t . v-OPD uses the detached baseline

$$\hat{b}_t = -D_{\text{KL}}(\bar{\pi}_\theta(\cdot | c_t) \| \bar{\pi}_{t^*}(\cdot | c_t)) \quad (7)$$

and advantage

$$a_t = r_t - \hat{b}_t = r_t + D_{\text{KL}}(\bar{\pi}_\theta \| \bar{\pi}_{t^*}). \quad (8)$$

The corresponding maximization gradient estimator is

$$\mathbb{E}_{y \sim \pi_\theta} \left[\sum_t a_t \nabla_\theta \log \pi_\theta(y_t | c_t) \right]. \quad (9)$$

Both r_t and $\hat{b}_t$ are detached. Top- K only approximates an action-independent control-variate baseline; it is neither a truncated-KL target nor a teacher selector. It therefore leaves the expected sampled-token OPD gradient unchanged.

C.2 Final OPD Configuration and Compute

The OPD run uses 4× NVIDIA H100 PCIe GPUs, takes about six wall-clock hours, and consumes about 25 GPU-hours. Its source alias is `slime-opd-best-skip-snapshot`. The 100-update budget and best+skip rule were fixed before final benchmark evaluation. The OPD coefficient and K are singleton settings; Table 8 reports the routing alternatives considered.

Category	Value	Category	Value
Student model	stage-2 RL	Routing policy	reward gate
Student checkpoint	step 1000	Gate statistic	best
Stage-0 teacher samples	2 per prompt	Rejected-gate action	skip
Stage-1 teacher samples	2 per prompt	Objective	v-OPD top- K
Prompt batch size	2	OPD coefficient	1.0
Agentic rounds	2	Top- K	16
Student samples	4 per prompt	Top- K mode	post-hoc
EDA timeout	360 s	Teacher context	65,536 tokens

Table 7: Final adaptive-OPD hyperparameters.

C.3 Routing Variants

The variants separate source selection from the action taken when a teacher does not beat the student:

- **Best comparison.** The student score is the best of its 4 rollouts; each teacher score is the best of its 2 rollouts. We select the higher-scoring teacher and apply OPD only if its score exceeds the student score. A rejected group is either skipped or trained with the RL fallback.
- **Median comparison.** We replace each best score above with the median rollout score. The higher-median teacher supplies OPD only when its median exceeds the student’s; otherwise the group uses skip or RL fallback.
- **Always best.** We select the teacher with the highest best rollout and always apply OPD, without a student-teacher gate.
- **Random teacher.** We uniformly sample one of the 2 teachers and always apply OPD.
- **Random source.** We uniformly sample the RL fallback, the stage-0 teacher, or the stage-1 teacher, each with probability 1/3. The selected source determines whether the group uses RL or OPD.
- **Always fallback.** Every group uses the RL objective and no OPD.

Routing source	Unbeaten-task action	Pass@1	Pass@5	Cov@1	Cov@5
RL base	–	85.5	89.2	95.9	97.5
Best teacher	RL fallback	87.5	90.4	95.3	97.4
Best teacher	skip	88.0	91.6	96.0	97.6
Always fallback	RL fallback	85.1	90.4	95.0	97.2
Always best	distill	86.3	91.6	95.3	97.7
Random teacher	distill	87.0	90.4	95.3	97.3
Random source	sampled	85.1	90.4	95.4	96.1
Median teacher	RL fallback	86.0	90.4	95.8	96.8
Median teacher	skip	86.7	91.6	95.7	96.5

Table 8: CVDP-ECov OPD routing results at the 100-update budget. The bold row is the one reported in Table 3.

C.4 Failure Handling and Expected Limitations

A valid rejected gate has complete scores and triggers skip. Missing, non-finite, truncated, or mismatched scores are integrity failures and use the safe RL fallback.

The method has four limitations. Saturated groups may yield no OPD gradient. EDA failures reduce supervision and add runtime variance. Execution reward is a noisy routing proxy, although multiple rollouts and strict gating reduce this effect. Finally, $K = 16$ may coarsen the control-variate estimate; it affects variance reduction, not the sampled-token objective.

D Further Analysis and Ablations

D.1 Model Merging

Merge configuration. All merges use the stage-0, stage-1, and stage-2 RL checkpoints at step 1000. Uniform Soup averages them with weights $(1/3, 1/3, 1/3)$. DARE-TIES and DELLA use each checkpoint once as the base at density $\rho = 0.5$. Per-tensor seeds hash the artifact-recorded label with the method, base, source, and tensor key.

Method	Base	Members	Pass@1	Pass@5	Cov@1	Cov@5
RL expert	–	stage-0	85.8	89.2	95.8	97.5
RL expert	–	stage-1	84.6	90.4	95.8	97.6
RL expert	–	stage-2	85.3	89.2	95.5	97.1
Oracle union (analysis)	–	stage-0, stage-1	88.9	91.6	97.0	97.8
Oracle union (analysis)	–	stage-1, stage-2	88.7	91.6	96.9	97.7
Oracle union (analysis)	–	stage-0, stage-2	90.8	92.8	96.8	97.7
Oracle union (analysis)	–	all three	90.8	92.8	97.2	97.8
Uniform Soup	–	all three	86.7	91.6	94.9	97.4
DARE-TIES	stage-0	all three	77.6	88.0	91.1	97.5
DARE-TIES	stage-1	all three	84.8	89.2	96.4	97.6
DARE-TIES	stage-2	all three	86.0	92.8	95.4	96.8
DELLA	stage-0	all three	83.6	92.8	95.2	97.5
DELLA	stage-1	all three	84.8	88.0	95.1	96.4
DELLA	stage-2	all three	85.8	90.4	96.1	97.4

Table 9: CVDP-ECov model-merging results. Bold values are the ones reported in Table 2; Oracle union is analysis-only and not a trained model.

For a non-base delta Δ , DARE-TIES uses:

$$m \sim \text{Bernoulli}(\rho), \quad \tilde{\Delta} = \frac{m\Delta}{\rho}. \quad (10)$$

DELLA instead sets

$$p = \min\left(1, \frac{\rho|\Delta|}{\text{mean}(|\Delta|)}\right), \quad m \sim \text{Bernoulli}(p), \quad \tilde{\Delta} = \frac{m\Delta}{p}. \quad (11)$$

Both methods take the elementwise sign consensus and average aligned nonzero deltas. Delta arithmetic uses FP32; outputs are cast to the base dtype, and non-floating tensors are copied from the base.